LLDD BE - Job 4 PrepareImpactStoreToIAS

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

เตรียมและส่งคำขอยอดขายไป IAS: สร้างไฟล์คำขอยอดขาย IAS/MIS แบบ durable ก่อนเปลี่ยนสถานะ W→P แล้วบันทึก transactional outbox เพื่อส่งซ้ำได้โดยไม่สร้างรายการซ้ำ

1.2 ขอบเขตงาน

- Main class/script: fgi.main.PrepareImpactStoreToIAS / FGI_ExportImpactStoreToAMS.sh
- Phase: B
- Output: AMS06001O (UTF-8)
- Estimate: 13 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	17 ชั่วโมง = implementation 13 + unit test 4 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	`PrepareImpactStoreToIAS` (`fcsJar/src/th/co/gosoft/fgi/main/PrepareImpactStoreToIAS.java`)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-2-ImportImpactStore.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-prepare-impact-store-to-ias` ใน `src/main.ts` ของ `SBP/srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-prepare-impact-store-to-ias INPUT='{"year":2026,"month":6}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"year":2026,"month":6}' sgi-prepare-impact-store-to-ias`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: ไม่รับ args — ใช้ปี/เดือนปัจจุบัน จาก `Calendar` ไปประกอบชื่อไฟล์ (`ftpFileTimePattern`) และแปลง พ.ศ.→ค.ศ. เมื่อ `year > 2100`
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่
ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`year`	number	ปีปัจจุบัน	ค.ศ. 4 หลัก	ของใหม่ — legacy ไม่มี ทำให้ rerun งวดย้อนหลังไม่ได้
`month`	number	เดือนปัจจุบัน	1-12	ของใหม่
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- ชื่อไฟล์ยังเป็น `AMS060010_yyyyMMddHHmm.txt` ที่สร้างจากเวลา ณ ตอนรัน — `year`/`month` ใช้คัดงวดของ candidate ไม่ใช่เปลี่ยนชื่อไฟล์

2.2 Input / Output

Stage	Contract for implementation
Input	FGI_IMPACT_STORE_SALES rows waiting for IAS sales data and EAI S3 bucket/prefix parameters.
Progress	query eligible stores, write outbound IAS request file, upload to EAI S3 outbound prefix, keep local backup, record success/failure and notification.
Output	IAS request file containing store/open-date pairs; run history includes generated file name and exported row count.

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	lock รายการ sales_request_status=W (FOR UPDATE SKIP LOCKED)
3	สร้าง temporary file และ validate record count (ยังไม่เปลี่ยน W→P)
4	fsync + atomic rename + SHA-256 (ไฟล์ต้อง durable ก่อนเริ่ม DB transaction)
5	transaction: update W→P + insert outbox READY (fail แล้ว rollback ทั้งสถานะและ outbox)
6	dispatcher ส่ง SFTP ด้วย secretRef/strict known_hosts (retry จาก outbox transaction เดิม)
7	จบ

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 4 ตัดสินว่า ร้านไหนพร้อมขอยอดขายจาก IAS/MIS แล้ว — ไม่ใช่ทุกแถวที่สถานะ `'W'` จะถูกส่ง เพราะมีเงื่อนไข อายุร้าน และระยะเวลารอหลังร้านใหม่เปิด คุมอยู่ และทั้งสองข้อนี้หายไปจากเอกสารเดิม

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
ร้านนี้พร้อมส่งขอยอดขายหรือยัง	`sgi_fgi_impact_stores.verify_status` · `.sales_request_status` + วันเปิดร้าน snapshot `sgi_fgi_impact_stores.opendate_i/_n` (ค่าที่ ALLMAP ส่งมา · มติ 2026-10-05 · แถว STA และแถวที่ยังไม่มีค่าใช้ `mas_store.open_date`) ของทั้งร้าน I (`impacted_store_code`) และร้าน N (`new_store_code`)	ต้องจริงครบ 4 ข้อ: (0) `verify_status = 'P'` (ผ่านกฎของ Job 2 มาแล้ว) (1) `sales_request_status = 'W'` (2) ร้านเก่าเปิดก่อนร้านใหม่อย่างน้อย 12 เดือน 15 วัน → open_date(I) <= (open_date(N) - 12 เดือน) - 15 วัน (3) วันนี้ต้องเลย `open_date(N) + 15 + 1` วันไปแล้ว	ไม่ครบ = ไม่ส่งรอบนี้ ค้างเป็น `'W'` ให้รอบถัดไปหยิบ — ไม่ใช่ error และไม่นับเป็น rejected
ส่งอะไรลงไฟล์	คอลัมน์เดียวกับข้างบน	1 บรรทัด = `impacted_store_code + '|' + to_char(open_date(N),'YYYYMMDD')` · เรียงด้วย `open_date(N), impacted_store_code`	จำนวนบรรทัดต้องเท่ากับจำนวน candidate ที่ล็อกไว้ ไม่เท่า = ยกเลิกทั้งรอบ
เปลี่ยนสถานะเมื่อไร	`sgi_fgi_impact_stores.sales_request_status`	'W' → 'P' หลังเขียนไฟล์ลงดิสก์สำเร็จ (fsync + atomic rename) แล้วเท่านั้น และอยู่ transaction เดียวกับ insert outbox	อัปโหลด S3 ล้มเหลว ห้ามย้อน 'P' → 'W' (จะสร้างไฟล์ซ้ำ) — ให้ outbox retry แทน

▲ ช่องว่างของ schema ที่ต้องปิดก่อน implement เงื่อนไขข้างบนได้จริง

แถวในตารางนี้ไม่ใช่ "ข้อควรระวัง" แต่เป็น ของที่ยังไม่มีในโครง 21 ตาราง — เขียนโค้ดตามเงื่อนไขด้านบนแล้วจะ compile ไม่ผ่าน/คิวรีพังทันที

#	สิ่งที่ขาด	ต้องทำอะไรก่อน
G4 ⧗ ยังค้าง	ไม่มีคอลัมน์วันเปิดร้านใน `sgi_*` เลย (`open_date` ไม่ปรากฏใน DDL ทั้ง 20 ตาราง) — และจะไม่เพิ่ม (วันเปิดร้านเป็น master ของระบบเดิม คัดลอกมาเก็บจะ stale)	เงื่อนไข 12 เดือน 15 วัน และ 16 วัน ต้อง join ออกไปที่ `mas_store.open_date` (schema `sps_store`) ทุกครั้ง — ต้องยืนยัน สิทธิ์อ่าน + index บน `mas_store.branch_id` ก่อน implement

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
ระยะห่างอายุร้าน	12 เดือน 15 วัน	`FgiConstant.INTERVAL_MONTH = 12` · `INTERVAL_DAY = 15`
ระยะรอหลังร้านใหม่เปิด	15 + 1 = 16 วัน	`TRUNC(SYSDATE) > (TRUNC(OPENDATE_N + 15) + 1)`
สถานะของ `sales_request_status`	`W` = รอส่งขอ · `P` = ส่งขอแล้วรอไฟล์ตอบกลับ · `Y` = ได้ยอดแล้ว · `E` = ผิดพลาด	`CHECK` ใน DDL

SQL คัด candidate (แปลงจาก `ImportStoreJdbc.getPrepareImpactStoreToIASList` บรรทัด 99-115)

ค่า 12 / 15 ต้องอ่านจาก config ไม่ hardcode ในคิวรี แต่ ค่าตั้งต้นต้องเท่าระบบเดิม
ไม่งั้นจำนวนร้านที่ส่งขอจะไม่ตรงกับของเดิมตอนเทียบ UAT

```
SELECT fis.id,
      fis.impact_process_id,
      fis.impacted_store_code,
      fis.new_store_code,
      fis.impact_month,
      fis.impacted_store_code || '|' || to_char(ms_n.open_date, 'YYYYMMDD') AS line
FROM sgi_fgi_impact_stores fis
JOIN mas_store ms_i ON ms_i.branch_id = fis.impacted_store_code -- schema sps_store (อ่านอย่างเดียว)
JOIN mas_store ms_n ON ms_n.branch_id = fis.new_store_code
WHERE fis.verify_status = 'P' -- ผ่านกฎ DENY/ON_PROCESS ของ Job 2 มาแล้วเท่านั้น
  AND fis.sales_request_status = 'W'
  AND ms_i.open_date <= date_trunc('day', ms_n.open_date - INTERVAL '12 months' - INTERVAL '15 days')
  AND date_trunc('day', CURRENT_DATE) > date_trunc('day', ms_n.open_date + INTERVAL '15 days') + INTERVAL '1 day'
ORDER BY ms_n.open_date, fis.impacted_store_code
FOR UPDATE OF fis SKIP LOCKED;
```

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 16 7-16 * *	แก้ไขได้	รันวันที่ 7-16 เวลา 16:00
IAS SFTP endpoint alias	ias-sales-request	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	host/port resolve จาก environment; credential ใช้ secretRef และ strict known_hosts
Secret reference	secret/sbpgi/interfaces/ias	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ห้ามเก็บ password/private key ใน job_configs
Output staging path	/data/sbpgi/outbox/ias	แก้ไขได้	ต้องรองรับ temp file, fsync และ atomic rename

2.6 หลักฐานที่ต้องเก็บตอนรัน

Evidence	Job-specific value	Acceptance
Input identity	FGI_IMPACT_STORE_SALES rows waiting for IAS sales data and EAI S3 bucket/prefix parameters.	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	IAS request file containing store/open-date pairs; run history includes generated file name and exported row count.	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	ชื่อไฟล์ deterministic จาก period+runId และ UNIQUE(data_name,direction,business_key,period_key); outbox retry ใช้ transaction เดิม ไม่สร้าง request ซ้ำ	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ
Transaction proof	สร้างไฟล์ temp, fsync, atomic rename และคำนวณ checksum ให้สำเร็จก่อน; จากนั้น transaction เดียว lock W, update W→P และ insert outbox READY; ห้าม commit W→P ก่อนมี durable file	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ

Evidence	Job-specific value	Acceptance
Security proof	สิทธิ์เขียน EAI S3 ใช้ IAM role ของ AWS Batch หรือ secretRef=secret/sgi/interfaces/eai-s3; จำกัดสิทธิ์เฉพาะ prefix ขาออก interface/out/AMS/ (PutObject เท่านั้น) และห้าม editable access key ในหน้าจอ/ไฟล์ config · ● ห้ามใช้ `interface/out/IAS/` ซึ่งเป็นโฟลเดอร์ของทีมอื่น (มี upn_name_subarea_*.txt อยู่จริง)	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

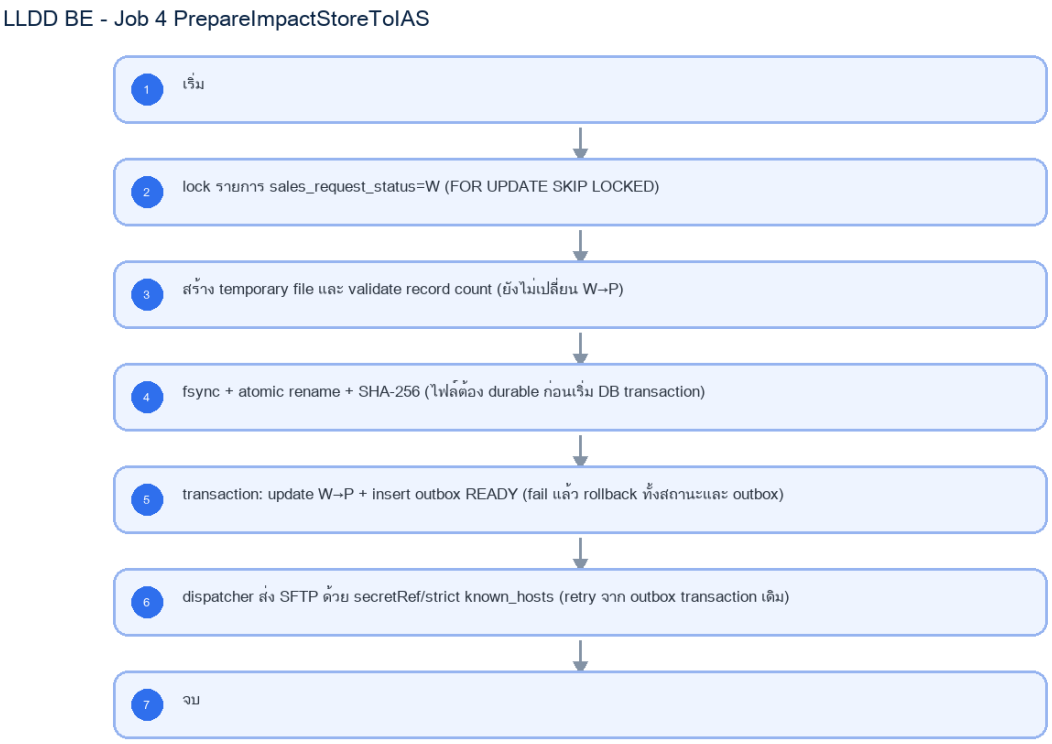
Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/PrepareImpactStoreToIAS.java	28-243	Legacy main entrypoint, file generation, upload, backup, notification.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java	99-115	Query FGI_IMPACT_STORE_SALES rows eligible for IAS request.

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

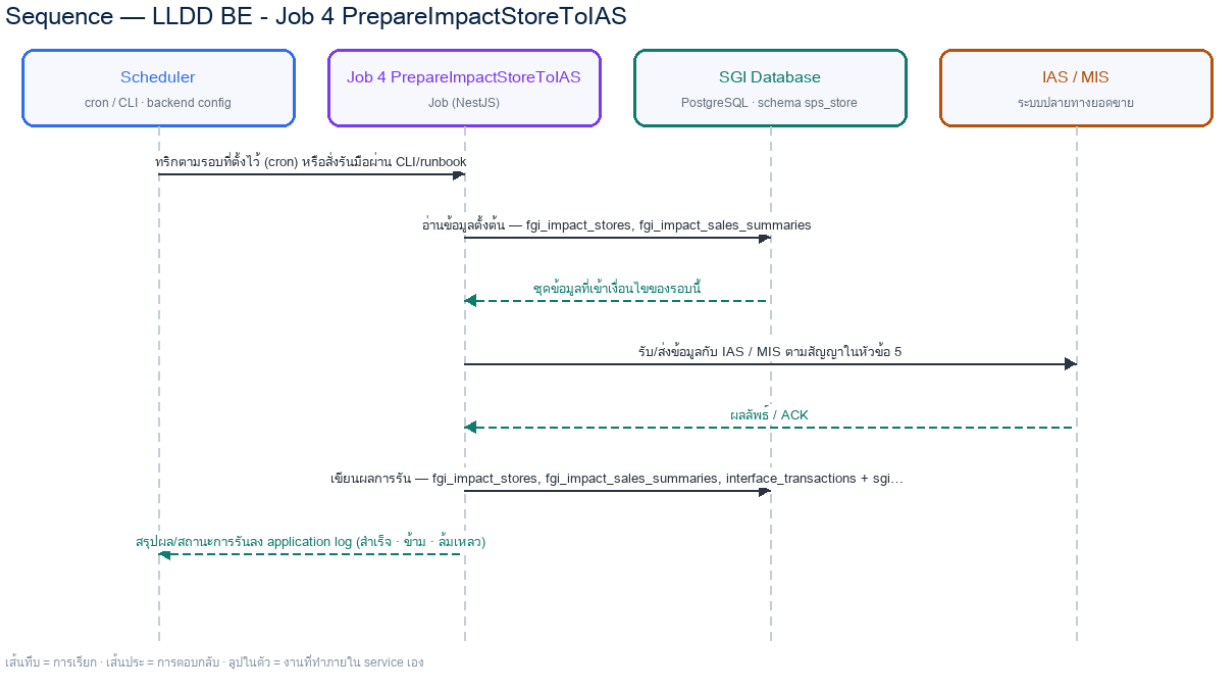
2.8 Job 4 Atomic File / Outbox Sequence

Order	Required action	Failure behavior
1	lock candidate W ด้วย FOR UPDATE SKIP LOCKED และสร้าง payload ใน memory	validation fail: rollback lock; สถานะยัง W
2	เขียน temporary file, fsync, atomic rename และคำนวณ SHA-256	write/rename/checksum fail: ลบ temp; สถานะยัง W; ไม่สร้าง outbox
3	transaction เดียว update W→P และ insert sgi_interface_transactions/outbox READY	DB fail: rollback W→P และ outbox; durable file คงไว้ให้ cleanup/reconcile โดย checksum
4	dispatcher อ่าน READY แล้วอัปโหลดขึ้น EAI S3 (prefix ขาออก); compare checksum ก่อนส่ง	อัปโหลด fail: outbox ยัง READY/FAILED_RETRY; ห้ามเปลี่ยน candidate กลับ W เพื่อไม่ให้สร้างไฟล์ซ้ำ
5	publish สำเร็จ mark SENT + outbox_status = PUBLISHED; ได้ publisher confirm จึง CONFIRMED + status = COMPLETED	ใช้ transaction id เดิมตลอด lifecycle

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 4 PrepareImpactStoreToIAS



รูปที่ 2: Sequence diagram: LLDD BE - Job 4 PrepareImpactStoreToIAS

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
fgi_impact_stores	R/W	lock candidate W และเปลี่ยนเป็น P หลัง durable file สำเร็จเท่านั้น
fgi_impact_sales_summaries	R/W	สร้าง/ผูกหัวสรุปยอดขายใน transaction
interface_transactions	W	transactional outbox READY/SENT/ACKED พร้อม checksum และ idempotency key
job_run_histories	W	run status และ reconcile count

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-4` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-4-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-4-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-04-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (`BATCH_START`/`BATCH_END` + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

33 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

การเตรียมชื่อไฟล์

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	สร้างชื่อไฟล์จากเวลาปัจจุบันปกติ	เวลาปัจจุบัน = 17/05/2026 15:30	ได้ชื่อไฟล์ `AMS060010_20260517 1530.txt`
UT-02	เดือน/วัน/ชั่วโมง/นาทีที่เป็นเลขตัวเดียวต้อง pad เป็น 2 หลัก	เวลาปัจจุบัน = 03/01/2026 05:07	ได้ `AMS060010_202601030507.txt`

การเตรียมข้อมูลยอดขาย (insert เข้าคิว)

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-03	มีคู่ร้านที่เข้าเงื่อนไข (`flag_verify='P'`, `create_by='ALM'`) และยังไม่อยู่ในคิว	คู่ร้านใหม่ที่ตรงเงื่อนไข	insert เข้าคิวสำเร็จด้วยสถานะ `flag_verify='W'`
UT-04	คู่ร้าน+งวดเดียวกันมีอยู่ในคิวแล้ว	คู่ร้านที่ query ยอดคืนมาแต่ตรงกับที่มีอยู่แล้วใน `sgi_fgi_impact_sales_summaries`	ไม่ insert ซ้ำ
UT-05	คู่ร้านอยู่ใน process ที่ยัง active (`flag_action='Y'`)	คู่ร้านที่ `storecode_i` ตรงกับ process ที่ active	ไม่ insert
UT-06	มีหลายแถวซ้ำ `storecode_i+เดือน+ปี`	หลาย record คู่กันในกลุ่มเดียวกัน	insert เฉพาะแถวที่มีลำดับความสำคัญสูงสุด (เรียงตามวันเปิดร้านใหม่แล้วตามรหัสร้านใหม่)
UT-07	insert เกิด error จากฐานข้อมูล	mock DB error ระหว่าง insert	throw error ออกไปทันที ไม่กลืนไว้เงียบๆ สรุปผลเป็นล้มเหลว

การอ่านรายการที่พร้อมส่งออก

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-08	มีรายการที่ผ่านเงื่อนไขระยะเวลาครบทั้งสองข้อ	record ที่เปิดก่อนร้านใหม่ >= 12 เดือน 15 วัน และผ่านไปแล้ว >16 วันจากวันเปิดร้านใหม่	ได้ record นั้นกลับมาในผลลัพธ์
UT-09	ยังไม่ผ่านเงื่อนไขระยะเวลาข้อใดข้อหนึ่ง	record ที่เพิ่งเปิดร้านใหม่ไม่ถึง 16 วัน	ไม่ได้ record นั้นกลับมา
UT-10	record ไม่ได้อยู่ในสถานะรอส่งออก	record ที่ `flag_verify='P'` อยู่แล้ว	ไม่ได้ record นั้นกลับมา
UT-11	query เกิด error	mock DB error ระหว่าง query	throw error ออกไปทันที ไม่คืนค่าว่างเปล่าแทน
UT-12	ผลลัพธ์เรียงตามลำดับที่กำหนด	หลาย record ที่วันเปิดร้านใหม่/รหัสร้านต่างกัน	เรียงตามวันเปิดร้านใหม่ก่อน แล้วตามรหัสร้านที่ถูกกระทบ
UT-13	ไม่มีรายการที่พร้อมส่งออก	ผลลัพธ์ query เป็น list เปล่า	สรุปผลเป็น NO_DATA ไม่เปิด transaction ไม่เขียนไฟล์

การแปลงข้อมูลและสร้างเนื้อหาไฟล์

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-14	มีหลายแถว	3 record	เนื้อหาไฟล์คั่นแต่ละแถวด้วย newline, แถวสุดท้ายไม่มี newline ต่อท้าย
UT-15	สร้าง record ประวัติจากแต่ละแถว	record ที่มีรหัสร้าน/เดือน/ปีถูกต้อง	ได้ record ประวัติที่มีชื่อประเภทข้อมูล, รหัสอ้างอิงธุรกรรม, และชื่อไฟล์ที่จะ export ตรงกัน
UT-16	record ประวัติไม่ได้กำหนดรหัสผลตอบรับและวันที่ตอบรับ	record ใดๆ	เมื่อ insert ลงฐานข้อมูล คอลัมน์ `return_code` และ `receive_date` เป็นค่าว่างเปล่าเสมอ
UT-17	รหัสร้านไม่ใช่ตัวเลขล้วน (เช่นมีตัวอักษรปน)	รหัสร้าน = `"AB123"`	reject เฉพาะแถวนั้นพร้อมเหตุผล แล้วอ่านแถวถัดไปต่อ ไม่ทำให้ทั้ง batch ล้มเหลว

การเขียนไฟล์และ backup บน S3

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-18	เขียนไฟล์ขึ้น S3 output prefix สำเร็จ	เนื้อหาไฟล์พร้อมส่ง	putObject สำเร็จ ทำ copy ไป backup prefix ต่อ
UT-19	เขียนไฟล์ขึ้น S3 ไม่สำเร็จ	mock S3 putObject ล้มเหลว	throw InfrastructureError, ไม่เปิด transaction, ไม่อัปเดตสถานะ, ไม่บันทึกประวัติ, สรุปผลเป็นล้มเหลว
UT-20	copy ไป backup prefix สำเร็จ	putObject สำเร็จก่อนหน้า	copyObject สำเร็จ ไปทำ transaction ต่อ
UT-21	copy ไป backup prefix ล้มเหลว	mock S3 copyObject ล้มเหลว	throw InfrastructureError, ไฟล์ที่ output prefix ยังคงอยู่ (ไม่ถูกลบ), ไม่อัปเดตสถานะ, ไม่บันทึกประวัติ, สรุปผลเป็นล้มเหลว

การบันทึกข้อมูล / Transaction

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-22	อัปเดตสถานะและบันทึกประวัติสำเร็จทั้งคู่	เขียนไฟล์และ backup สำเร็จก่อนหน้า	commit transaction, สถานะเปลี่ยนเป็นอยู่ระหว่างดำเนินการ, ประวัติถูกบันทึกครบ, สรุปผลสำเร็จ
UT-23	อัปเดตสถานะเกิดขึ้นหลังยืนยันว่าเขียนไฟล์และ backup สำเร็จเท่านั้น	mock เขียนไฟล์ล้มเหลวก่อนถึงขั้นตอน transaction	ไม่มีการอัปเดตสถานะใดๆเกิดขึ้นเลย (ต่างจากพฤติกรรมเดิมที่อัปเดตไปก่อน)
UT-24	เกิด error ระหว่าง update สถานะ	mock DB error ระหว่าง update	throw error ออกจาก transaction ทันที ไม่มีการรัน SQL ซ้ำในบล็อก catch, rollback ทั้ง update และ insert ประวัติ, สรุปผลเป็นล้มเหลว
UT-25	เกิด error ระหว่าง insert ประวัติ	mock DB error ระหว่าง insert ประวัติ	rollback ทั้ง update สถานะที่ทำไปก่อนหน้าในรอบเดียวกันด้วย, สรุปผลเป็นล้มเหลว
UT-26	บันทึกประวัติสำเร็จโดยไม่มีรายการ reject เลย	ทุกแถวแปลงรหัสร้านเป็นตัวเลขได้หมด	จำนวนที่บันทึกประวัติเท่ากับจำนวนแถวที่พร้อมส่งออกทั้งหมด

ผลลัพธ์มาตรฐานและ Error

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-27	เกิด error กลางทาง (เช่น database error ระหว่าง transaction)	mock error หลังผ่านขั้นตอนเขียนไฟล์/backup ไปแล้ว	ผลลัพธ์ FAILED ยังคงชื่อไฟล์ที่พยายาม export, เวลาเริ่ม/จบ และตัวเลขนับที่ทราบก่อนเกิด error ไว้ครบ
UT-28	error message ที่แสดงใน result ต้อง sanitize	error ที่มี connection string/credential อยู่ใน stack trace เดิม	ข้อความ error ที่แสดงไม่มีข้อมูลอ่อนไหวหลุดออกมา
UT-29	ผลลัพธ์เป็นสำเร็จหรือไม่มีข้อมูล	ทำงานจบปกติ	exit code = `0`
UT-30	ผลลัพธ์เป็นล้มเหลว	เกิด error ณ จุดใดก็ตาม	exit code = `1`

การแจ้งเตือน / Notification

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-31	หา template และส่งอีเมลสำเร็จ	mock ทั้งสองขั้นตอนสำเร็จ	อีเมลถูกส่งด้วยเนื้อความที่ render จากผลลัพธ์จริง ไปยังผู้รับที่ตั้งค่าไว้ใน config
UT-32	หา template ไม่พบ	mock query template คืนค่าว่าง	log ไว้เท่านั้น ไม่ throw error, ไม่กระทบผลลัพธ์ที่ commit แล้ว

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-33	ส่งอีเมลผ่าน SMTP ล้มเหลว	mock SMTP ล้มเหลว	log ไว้เท่านั้น, ผลลัพธ์ที่ได้ก่อนหน้าไม่เปลี่ยนเป็นล้มเหลว, ไม่มีการ retry

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. PrepareImpactStoreToIAS.java — lines 28-39

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/PrepareImpactStoreToIAS.java
Original lines	28-39
Responsibility	Legacy main entrypoint, file generation, upload, backup, notification.

```
public class PrepareImpactStoreToIAS {

private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");
private static FtpBean config = (FtpBean) context.getBean("fgiPrepareImpactStoreToIAS");
private static ImpactStoreService storeImpactService = (ImpactStoreService) context.getBean("storeImpactService");
private static ExportService exportService;
    private static final String NEW_LINE = System.getProperty("line.separator");

public static void main(String[] args) {
LogUtils.info(PrepareImpactStoreToIAS.class,"============****** Start => export FGI_IMPACT_STORE_SALES to MIS ******============");
/*****************************************************************************
* STEP 01 : อ่านข้อมูลเวลา, วัน, เดือน, ปีที่ทำข้อมูล โดยกำหนดให้ ข้อมูลเป็นวันและเวลาปัจจุบัน
```

J1. PrepareImpactStoreToIAS.java — lines 232-243

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/PrepareImpactStoreToIAS.java
Original lines	232-243
Responsibility	Legacy main entrypoint, file generation, upload, backup, notification.

```
LogUtils.info(ImportController.class,"----- " + fileName + " : backup= [fail] -----");
}
}

public ExportService getExportService() {
return exportService;
}

public void setExportService(ExportService exportService) {
this.exportService = exportService;
}
}
```

J2. ImportStoreJdbc.java — lines 99-115

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	99-115
Responsibility	Query FGI_IMPACT_STORE_SALES rows eligible for IAS request.

```
public List<Map<String, String>> getPrepareImpactStoreToIASList(){
try{
StringBuilder sqlCommand = new StringBuilder();
sqlCommand.append(" SELECT STORECODE_I, OPENDATE_N, MONTH, YEAR, STORECODE_I || '|' ||TO_CHAR(OPENDATE_N, 'YYYYMMDD') AS RESULT ");
sqlCommand.append(" FROM FGI_IMPACT_STORE_SALES ");
sqlCommand.append(" WHERE FLAG_VERIFY = '"+FgiConstant.FLAG_VERIFY_WAIT+"' ");
sqlCommand.append(" AND OPENDATE_I <= TRUNC(ADD_MONTHS(OPENDATE_N, -"+FgiConstant.INTERVAL_MONTH+")-"+FgiConstant.INTERVAL_DAY+")
");
sqlCommand.append(" AND TRUNC(SYSDATE) > (TRUNC(OPENDATE_N + "+FgiConstant.INTERVAL_DAY+") + 1) ");
sqlCommand.append(" ORDER BY OPENDATE_N, STORECODE_I ");

List impactStoreList = jdbcTemplate.queryForList(sqlCommand.toString());
LogUtils.info(getClass(),"----- query FGI_IMPACT_STORE_SALES for export : " + impactStoreList.size() + " -----");
return impactStoreList;
}catch(Exception e){
LogUtils.error(getClass(),e);
return null;
}
```